RESUMEN DE PAPER · GENERACIÓN PROCEDURAL DE NIVELES 2D

Launchpad: un generador de niveles basado en ritmo para plataformas 2D

Launchpad: A Rhythm-Based Level Generator for 2-D Platformers

Autores	Gillian Smith, Jim Whitehead, Michael Mateas, Mike Treanor, Jameka March, Mee Cha (Expressive Intelligence Studio, UC Santa Cruz)

Publicación	IEEE Transactions on Computational Intelligence and AI in Games, vol. 3, n.º 1, marzo 2011, pp. 1–16 · DOI 10.1109/TCIAIG.2010.2095855
Palabras clave	Inteligencia artificial, videojuegos, diseño de niveles, gramáticas multinivel, generación procedural de contenido (PCG)
Técnica central	Gramática de dos niveles (ritmo → geometría) + generar-y-probar con críticos

Resumen

Launchpad es un generador autónomo de niveles para juegos de plataformas 2D que se apoya en un **modelo formal del diseño de niveles** del género. Los niveles se arman a partir de segmentos cortos llamados **grupos de ritmo** (*rhythm groups*), que se producen con un enfoque gramatical de dos niveles: primero se genera un ritmo de acciones del jugador y luego se "realiza" ese ritmo como geometría. Los grupos se encadenan para formar niveles completos, que se evalúan con heurísticas de diseño (críticos) para elegir el mejor. El diseñador controla la generación mediante parámetros que influyen en el ritmo y la geometría. Como trabaja con componentes básicos comunes a muchos plataformas (y no con trozos grandes hechos a mano), el contenido que hay que autorizar manualmente es mínimo; además, **todos los niveles generados son jugables por construcción**.

Contribuciones principales

- Un **modelo formal** de los componentes y la estructura de los niveles de plataformas.
- Un **generador autónomo y guiado por el diseñador** basado en ese modelo (Launchpad).
- Métodos para **evaluar y visualizar el rango expresivo** (*expressive range*) de un generador de contenido, aplicados a Launchpad.

I. Introducción

Un nivel es el "contenedor del gameplay": el espacio donde el jugador explora y domina las mecánicas. Aunque es clave para el éxito de un juego, la ciencia detrás del diseño de niveles está poco formalizada. Con el auge de la generación procedural (como área de investigación y como forma de abaratar la producción de contenido), los autores sostienen que hace falta modelar formalmente cómo reconocer y diseñar buenos niveles, para ir más allá de los generadores que solo "cosen" grandes trozos de contenido hecho por humanos.

Los plataformas 2D (p. ej. *Super Mario World*, *Sonic the Hedgehog*) son ideales para investigar: reglas simples, diseño de niveles emergentemente complejo y una experiencia del jugador fuertemente determinada por el nivel, sin la complicación de narrativa o mecánicas sofisticadas.

La idea clave es el **ritmo**: la temporización y repetición de acciones del jugador. Launchpad lo captura con una gramática de dos niveles (un generador de ritmo y, sobre él, un generador de geometría), de modo que el ritmo deseado siempre está presente, sin importar la representación

geométrica elegida. Los parámetros del diseñador definen la trayectoria general del nivel, los tipos y frecuencias de componentes y la distribución de coleccionables.

Finalmente, el paper introduce el concepto de **rango expresivo** de un generador: mostrar unos pocos ejemplos y estadísticas de velocidad no basta para entender qué tipo de contenido produce ni cómo cambia según los parámetros. Proponen métricas externas y técnicas de clustering para visualizar ese espacio y descubrir sesgos inesperados.

II. Trabajo relacionado

A. Análisis de dominio para el diseño de niveles

Los libros de diseño de juegos tratan el diseño de niveles de forma general (distribución espacial, atmósfera, repetición) y dedican muy poco a géneros específicos. Excepciones relevantes: el análisis de Nelson sobre *Breakout* (descomponer niveles en componentes y dar reglas de composición, que es justamente el enfoque adoptado aquí), los patrones de diseño en niveles FPS de Hullett y Whitehead, y los análisis de plataformas de Boutros y Dormans. A estos últimos les falta un modelo formal de estructura y desafío, requisito para un generador que no dependa de piezas hechas a mano. Byrne y Nicollet destacan el **desafío** (y su temporización) como el núcleo del disfrute, lo que inspira la segmentación en grupos de ritmo.

B. Generación procedural de contenido

Se repasa la PCG desde *Rogue* (1980) y los *roguelikes* (p. ej. *Diablo*), que ensamblan trozos grandes con reglas simples y dan poco control al usuario. *Spelunky* combina roguelike y plataformas con piezas fijas en una grilla; funciona porque el jugador puede modificar el nivel (bombas, cuerdas), pero sería inadecuado para niveles estilo Mario. Otros trabajos cercanos:

- **Dormans**: generación en dos capas para juegos de acción-aventura (misiones con gramática → espacio con gramáticas de forma). Comparte con Launchpad la idea de una capa abstracta basada en acciones del jugador.
- **Compton y Mateas**: también dividen el nivel en secciones, pero el ritmo queda implícito en la geometría; Launchpad separa explícitamente ritmo y geometría y ofrece parámetros de control.
- **Mawhorter y Mateas** (*occupancy-regulated extension*): niveles muy variados y con múltiples caminos, a costa de no garantizar jugabilidad y de una alta carga de autoría.
- **Hullett y Mateas**: planificación HTN para generar escenarios de rescate a partir de una estructura existente.
- PCG personalizada: pistas de carreras, armas evolutivas (*Galactic Arms Race*), niveles Mario adaptados; y **Polymorph**, que usa la técnica de Launchpad para ajuste dinámico de dificultad.
- Enfoques gramaticales fuera de juegos (*Instant Architecture*, *Tableau Machine*); este último inspiró el uso de críticos con generar-y-probar.

III. Deconstruyendo los juegos de plataformas

Para construir un generador se necesita un modelo formal: componentes básicos y reglas de combinación. Los autores analizan *Super Mario World*, la saga *Sonic*, *Yoshi's Island DS* y *Donkey Kong Country 2*. Son estilos distintos (Mario: un solo camino y recompensa ligada al riesgo; Sonic: múltiples caminos y estilo *speed run*; DKC2: énfasis en áreas secretas), pero comparten desafíos de destreza y un sentido de ritmo en su ritmo de juego. La Fig. 1 resume el marco jerárquico resultante.

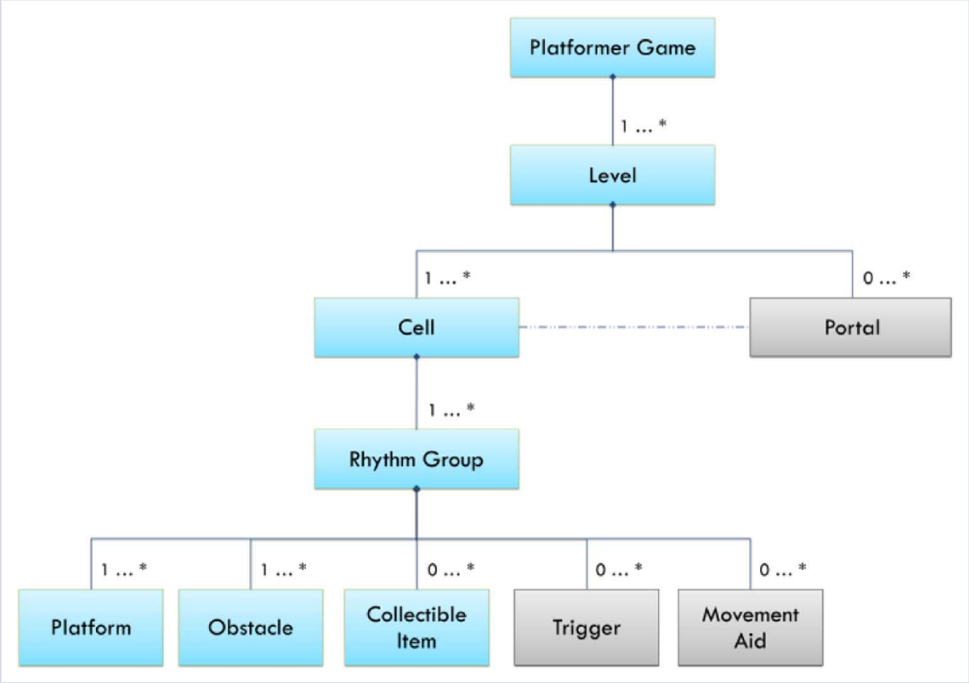


Figura 1. Marco jerárquico de niveles. Juego → Nivel → Celdas (y Portales) → Grupos de ritmo → Plataformas, Obstáculos, Coleccionables, Disparadores y Ayudas de movimiento. En azul, los elementos implementados en Launchpad.

A. El avatar

Es el personaje que controla el jugador. Aunque a veces hay varios (Diddy y Dixie en DKC2), normalmente se controla uno a la vez, y sus diferencias pueden cambiar la física (en *Yoshi's Island DS*, cada bebé otorga habilidades distintas). Sus capacidades se aproximan con un modelo físico simple: control amplio del movimiento horizontal y limitado del vertical (saltar, agacharse).

B. Componentes del nivel

Se categorizan por el rol que cumplen; un componente puede tener más de un rol (los bloques de ítems de Mario son coleccionables y también plataformas).

- **Plataformas**: cualquier superficie por la que el avatar puede caminar con seguridad. Tienen fricción, tamaño y pendiente, y pueden moverse.
- **Obstáculos**: todo lo que daña al avatar o interrumpe el flujo. Un *hueco* entre plataformas cuenta como obstáculo aunque no sea un objeto. Se superan eliminándolos o evitándolos.
- **Ayudas de movimiento**: escaleras, cuerdas, resortes, trampolines; modifican temporalmente la dirección o velocidad. Launchpad implementa el **resorte**.
- **Coleccionables**: monedas, anillos, power-ups. Dan recompensa, pero también **guían** al jugador (p. ej. monedas en la cima del salto, Fig. 2).
- **Disparadores** (*triggers*): objetos o acciones que cambian el estado del nivel (el botón “P” de Mario, Fig. 3; botones temporizados de Yoshi). Aportan un elemento de puzle (p. ej. *Shift*).



Figura 2. Uso de coleccionables (Super Mario World). La moneda Yoshi, de mayor recompensa, está en la zona de mayor riesgo; las otras monedas marcan el camino ideal.



Figura 3. Disparadores. El botón "P" convierte bloques en monedas, a cambio de liberar a los enemigos que contienen.

C. El ritmo como representación estructural

El ritmo y el *pacing* son centrales para el disfrute y para la dificultad en juegos de destreza. Por eso los niveles se representan como una sucesión de **grupos de ritmo**: conjuntos cortos y no superpuestos de componentes que encapsulan una zona de desafío. Cada grupo tiene un *ritmo* de acciones (el compás con que el jugador ideal pulsa botones) y la *geometría* que corresponde a ese ritmo. En la Fig. 4 hay dos grupos aunque la geometría parezca repetirse: la ausencia de monedas en el punto de unión crea una pausa que rompe el ritmo y separa ambos grupos.

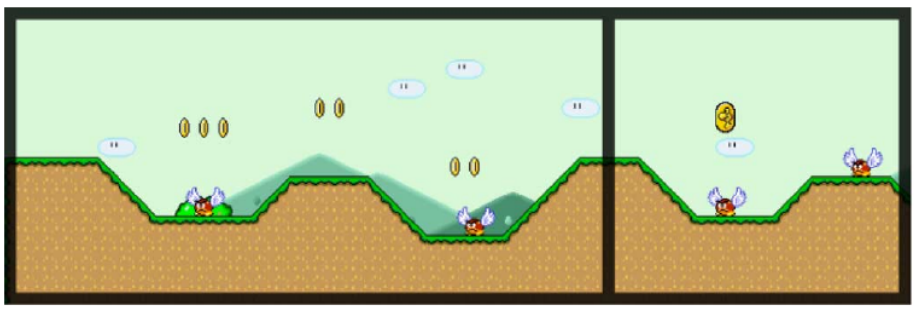


Figura 4. Ejemplo de grupos de ritmo en Super Mario World. Cada rectángulo negro es un grupo; los separa una plataforma de descanso donde no se requiere ninguna acción.

D. Elección del jugador y no linealidad

La no linealidad se modela con **celdas** (regiones de juego lineal y no superpuesto) y **portales** (transiciones entre celdas: entradas secundarias, cambios de camino, áreas secretas). En Sonic, cada punto donde se cruzan caminos es un portal. Catalogar celdas y portales permite analizar los distintos caminos, que pueden diferir en dificultad y recompensa.

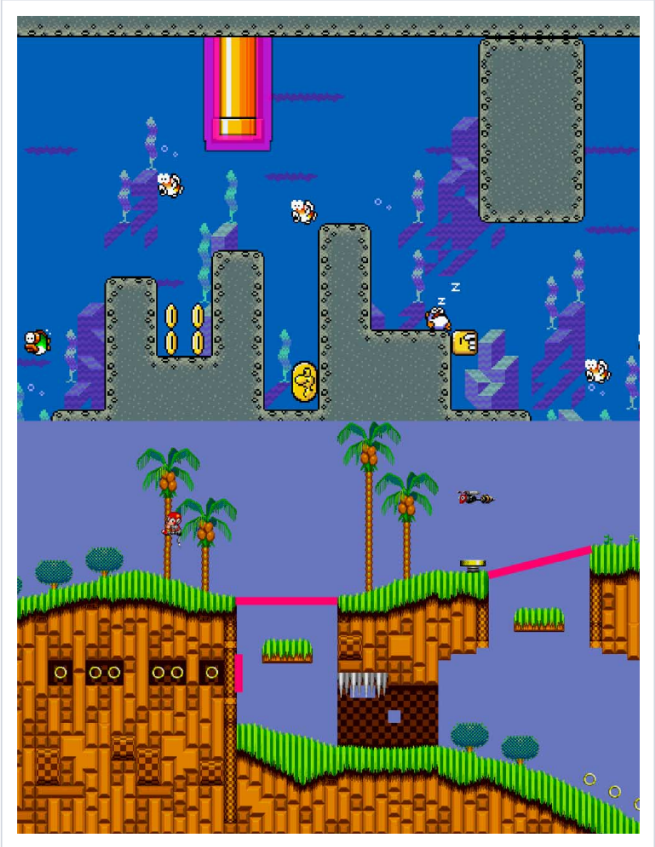


Figura 5. Celdas y portales. Arriba, Super Mario World: una celda con una tubería que actúa como portal. Abajo, Sonic 2: tres celdas (superior, inferior y zona secreta de monedas) conectadas por plataformas móviles que funcionan como portales.

IV. Construyendo niveles

Para simplificar, Launchpad genera **niveles de una sola celda** (sin disparadores, áreas secretas ni caminos múltiples, que quedan como trabajo futuro). Implementa plataformas planas o inclinadas (fijas o móviles en horizontal/vertical), obstáculos (enemigos a matar, enemigos a evitar como pinchos, *stompers* y huecos), resortes, y monedas que se colocan al final en una pasada global.

El flujo (Fig. 6) es: (1) generar un ritmo de “beats” (cada beat = una acción del jugador); (2) realizar ese ritmo como geometría con una gramática sujeta a restricciones físicas → un grupo de ritmo; (3) unir grupos con pequeñas plataformas seguras para formar niveles candidatos; (4) elegir el mejor candidato con **críticos**; (5) aplicar **pasadas globales** (monedas y suelo común). Cada etapa tiene parámetros controlables (Tabla I).

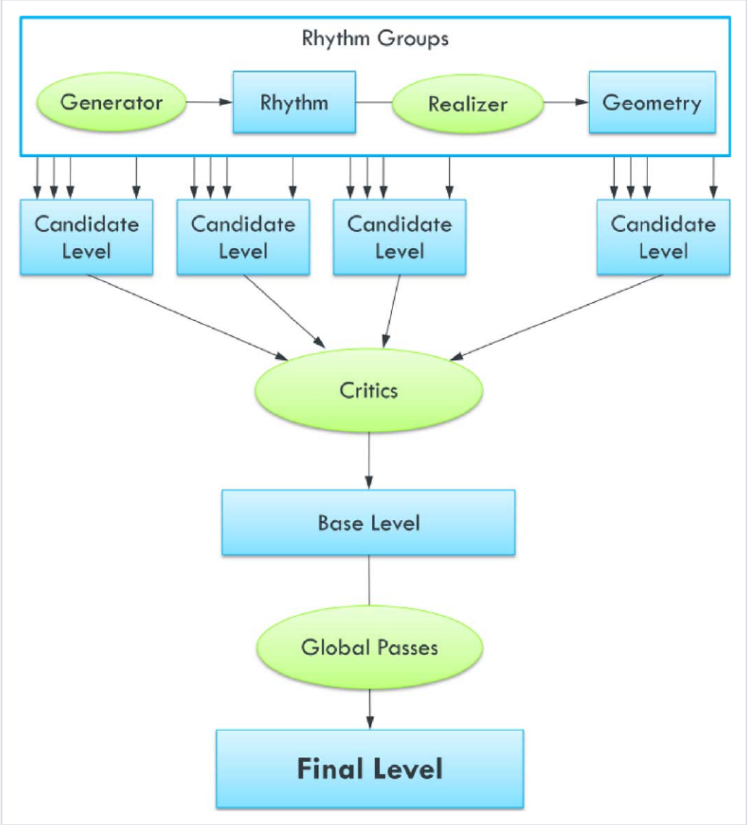


Figura 6. Arquitectura de Launchpad. Cajas azules: artefactos producidos; círculos verdes: componentes del generador (Generador → Ritmo → Realizador → Geometría → Niveles candidatos → Críticos → Nivel base → Pasadas globales → Nivel final).

Tabla I. Parámetros de Launchpad que el usuario puede manipular.

Etapa	Parámetro	Descripción
Ritmo	Tipo	“regular” (beats equiespaciados), “swing” (beat corto seguido de uno largo) o “aleatorio” (beats espaciados al azar).
	Densidad	Qué tan juntos están los beats: baja, media o alta.
	Longitud	Duración del ritmo: 5, 10, 15 o 20 segundos.
	Prob. de acciones	Probabilidad de una acción de salto vs. una de espera.
Geometría	Componentes de salto	Frecuencia deseada de cada geometría para un salto: subir o bajar con o sin hueco; enemigo; resorte; caída.
	Componentes de espera	Frecuencia deseada de cada geometría para una espera: *stomper*, plataforma móvil.
	Geometría repetida	Probabilidad de que un grupo de ritmo se repita inmediatamente.
Línea	Ecuación	Trayectoria que debe seguir el nivel, como segmentos de recta no superpuestos.
Crítico	Ponderación	Importancia relativa del crítico de distancia a la línea vs. el de frecuencia de componentes.
Monedas	N.º de monedas en grupo	Cantidad de monedas a colocar a lo largo de una plataforma.
	Longitud de plataforma	Umbral: plataformas más largas que esto reciben monedas.
	Prob. moneda sobre hueco	Probabilidad de poner una moneda sobre un hueco.

A. Generación del ritmo

Un ritmo tiene tres propiedades: **tipo**, **longitud** y **densidad**, elegidas según las probabilidades de la Tabla I (Fig. 7). Cada beat marca cuándo el jugador presiona un botón. Hay dos acciones, “mover” y “saltar”, cada una con tiempo de inicio y de fin (cuando se suelta el botón). La duración de un salto

importa porque mantener el botón más tiempo produce un salto más alto (0,25 s → salto corto; hasta 0,75 s → salto largo).

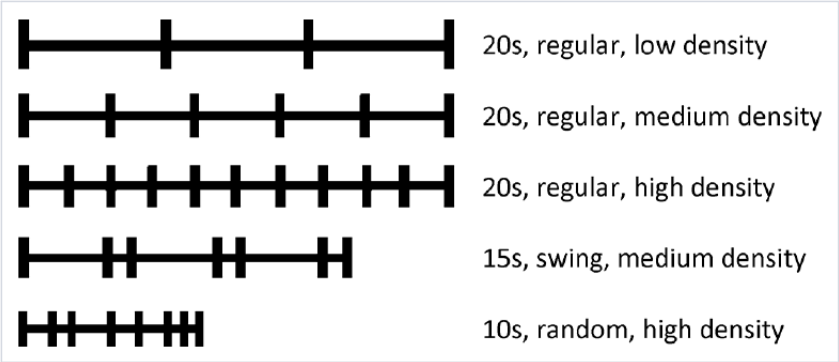


Figura 7. Ritmos de ejemplo según longitud, tipo y densidad. La línea indica la duración del ritmo y las marcas el inicio de cada acción.

Ejemplo usado en el paper (inicio y fin de cada acción, en segundos):

```
move  0     5        jump  6     6.5
jump  2     2.25     jump  8     8.5
jump  4     4.25
move  6     10
```

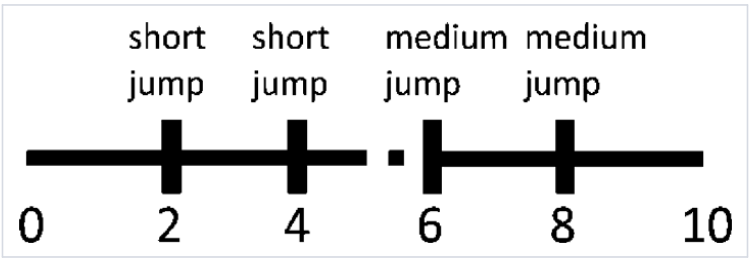


Figura 8. Ritmo con acciones del ejemplo: línea sólida = el avatar se mueve; línea discontinua = está detenido. Dos saltos cortos (2 s y 4 s) y dos medianos (6 s y 8 s).

B. Restricciones físicas

Un modelo físico balístico guarda el tamaño del avatar, su velocidad máxima, la velocidad inicial de salto, la altura alcanzable con salto corto/medio/largo, el tiempo en el aire de cada tipo de salto, la diferencia de altura entre plataformas que permite ese tiempo, el impulso de un resorte y las pendientes disponibles. No soporta doble salto ni salto en pared.

Con esto, los verbos del ritmo se convierten en una **lista de estados de movimiento** y una **cola de saltos**. Todo tiempo en que el jugador no se mueve se considera “espera”. Para el ejemplo: *Estados*: [5, moviendo], [1, esperando], [4, moviendo]; *Saltos*: [2, corto], [4, corto], [6, medio], [8, largo]. Como cada tipo de geometría consume distinto tiempo de aire (un hueco plano 0,5 s, un resorte 1,5 s), cada salto “consume” parte de la lista de estados (Fig. 9). Si un salto no termina antes del siguiente beat, esa geometría se descarta: así se **garantiza que todos los niveles sean jugables**.

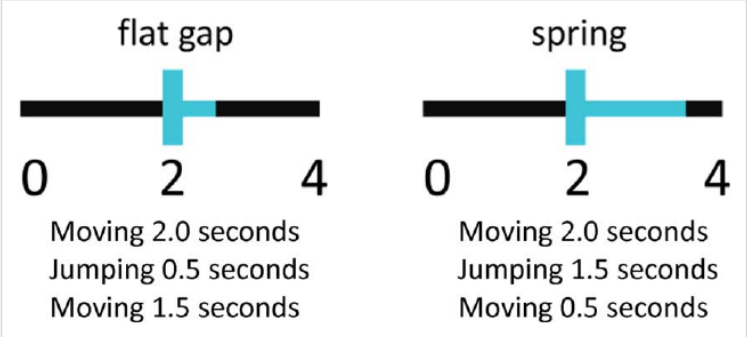


Figura 9. Creación de estados de movimiento: un salto sobre hueco plano y uno sobre resorte consumen distinto tiempo (zona azul = tiempo en el aire).

C. Realización de la geometría

Los estados no consumidos y los saltos en cola son los **no terminales** de la gramática de geometría (Fig. 10). Una espera por sí sola no tiene sentido (hay que esperar *algo*), así que generar geometría para ella requiere mirar hacia adelante: un patrón espera–movimiento produce un *stomper,* y espera–movimiento–espera una plataforma móvil. La elección está limitada por la física y ponderada por las frecuencias del diseñador. Un mismo ritmo admite muchas geometrías distintas (Fig. 11).

Moving → Sloped | flat_platform
Sloped → Steep | Gradual
Steep → steep_slope_up | steep_slope_down
Gradual → gradual_slope_up | gradual_slope_down

Jumping → flat_gap
| (gap | no_gap) (jump_up | Down | spring | fall)
| enemy_kill
| enemy_avoid
Down → jump_down_short | jump_down_medium
| jump_down_long

Waiting-Moving → stomper

Waiting-Moving-Waiting → moving_platform_horiz
| (moving_platform_vert_up | moving_platform_vert_down)

Figura 10. Gramática de generación de geometría. Los estados del jugador derivados del ritmo son los no terminales (Moving, Jumping, Down, Waiting-Moving, Waiting-Moving-Waiting).

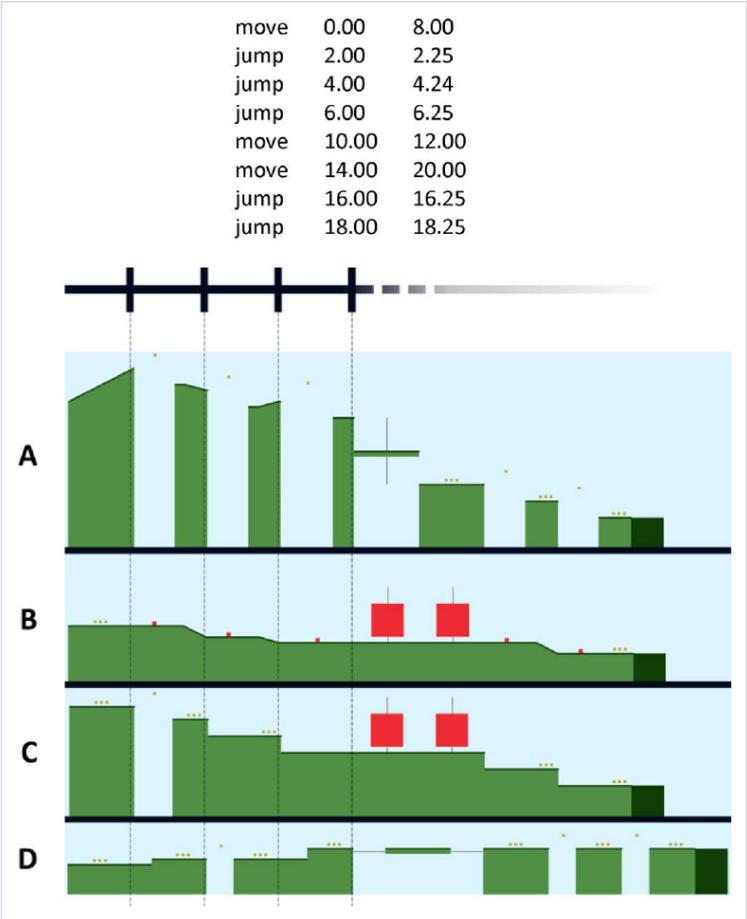


Figura 11. Cuatro interpretaciones geométricas (A–D) de un mismo ritmo. Cuadros rojos pequeños: enemigos a matar; grandes: *stompers*; plataformas sobre líneas verdes: plataformas móviles; bloque verde oscuro final: plataforma de unión con el siguiente grupo.

D. Críticos

Los niveles completos se forman uniendo grupos con una plataforma de descanso (importante para el *pacing*); un grupo puede repetirse antes del descanso según una probabilidad. Las gramáticas capturan bien restricciones locales (jugabilidad) pero tienden a **sobregenerar**; por eso se usan críticos con heurísticas globales para filtrar.

1) Crítico de distancia a la línea

El diseñador entrega una trayectoria por tramos que el nivel debería seguir (inicio, fin y dirección general). El mejor nivel minimiza la distancia promedio entre los extremos de las plataformas y la línea (Fig. 12). Refleja que los niveles de Mario y Sonic siguen recorridos regulares.

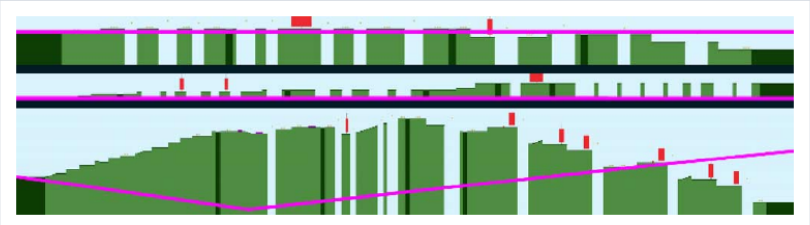


Figura 12. Puntuaciones del crítico de línea: niveles a distancia 1,75 (arriba), 5,08 (medio) y 23,06 (abajo) de la línea de control (rosado).

2) Crítico de frecuencia de componentes

Las frecuencias de la Tabla I solo se cumplen asintóticamente; en un nivel concreto pueden desviarse (como 10 lanzamientos de moneda que no dan 5 y 5). Este crítico aplica una **prueba chi-cuadrado de bondad de ajuste** y elige el nivel con el menor estadístico, es decir, el más

cercano al estilo deseado.

3) Combinación de críticos

Ambos pueden contradecirse (muchos resortes no encajan con una línea que solo baja). Se elige el nivel con la menor **suma ponderada** de ambos críticos.

E. Pasadas globales

Dos algoritmos razonan sobre todo el nivel: (1) **suelo común**: atar todas las plataformas a un plano de suelo según la plataforma más baja, para coherencia visual y para evitar caídas accidentales; (2) **decoración con monedas**, al estilo Super Mario World: grupos de monedas sobre plataformas largas sin otra acción, y una moneda en la cima de los saltos sobre huecos (recompensa el riesgo y guía la altura). Los autores notan que en juegos centrados en coleccionables (DKC) sería mejor colocarlos durante la generación de geometría.

V. Evaluación

Los ejemplos (Fig. 13) no muestran todo el rango de niveles posibles ni el impacto de los parámetros. Los autores llaman a esta cualidad **rango expresivo** y responden cuatro preguntas: cómo el diseño del algoritmo afecta lo que se puede producir; cómo medir los niveles; cómo influyen los parámetros; y cómo comparar niveles entre sí.

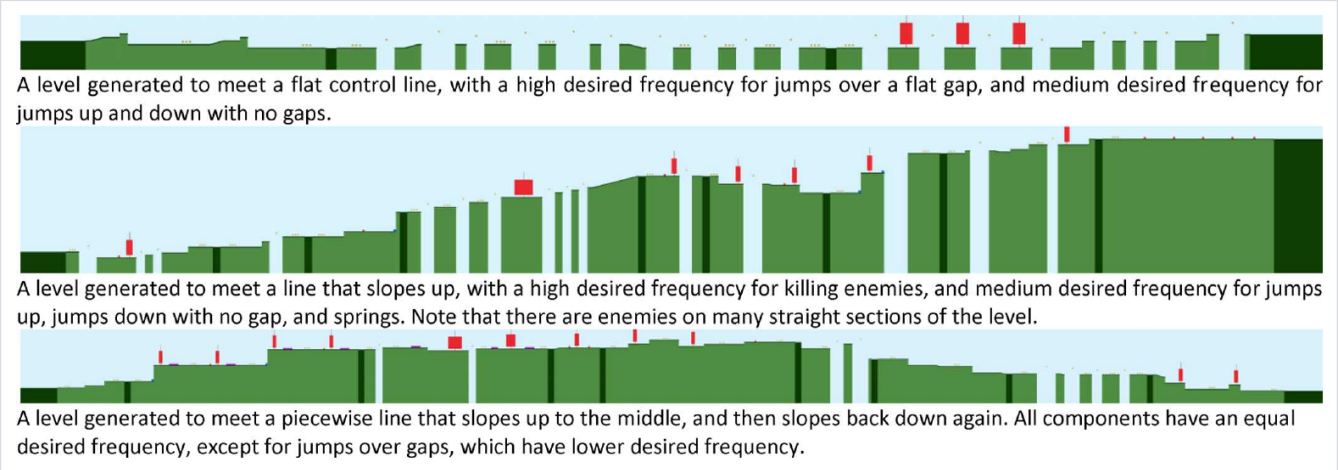


Figura 13. Niveles generados con distintos parámetros (cada uno es el mejor de 1000 candidatos): línea plana con muchos saltos sobre huecos; línea ascendente con muchos enemigos; línea que sube y luego baja con componentes equilibrados.

A. Implicancias del algoritmo

Por diseño, Launchpad produce niveles de **destreza** (temporizar movimientos frente a obstáculos) y no de exploración; no permite elegir caminos ni dar media vuelta. Por eso sus niveles favorecen un estilo *speed run*, lo que guía la elección de métricas.

B. Métricas de nivel

Las métricas deben medir propiedades **emergentes**, no repetir los parámetros de entrada. Se definen dos (más alto = mejor ajuste a la métrica):

1) Linealidad

Mide el "perfil" estético del nivel. Se ajusta una recta por regresión lineal sobre los puntos centrales de las plataformas, se suma la distancia absoluta de cada punto a la recta y se divide por el número de puntos; se normaliza a [0, 1] (1 = muy lineal). Pocos niveles tuvieron menos de 0,3. No debe confundirse con el crítico de línea: un nivel puede ser muy lineal y aun así alejarse de la línea de control (Fig. 15).

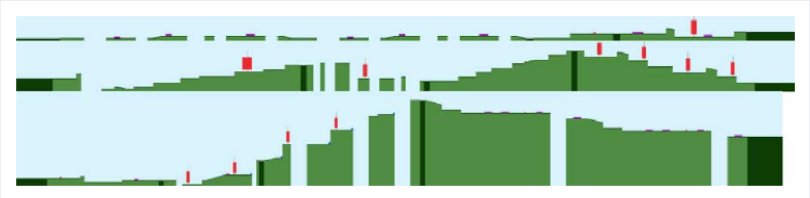


Figura 14. Linealidad decreciente: 0,93 (arriba), 0,60 (medio) y 0,00 (abajo).

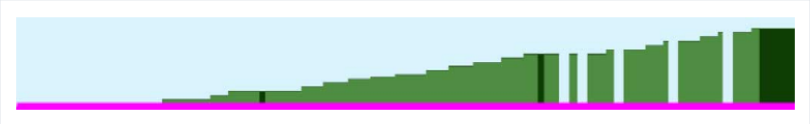


Figura 15. Linealidad vs. distancia a la línea: este nivel es muy lineal (sube uniformemente) pero puntúa mal en el crítico porque se aleja de la línea de control.

2) Indulgencia (leniency)

Estima qué tan "perdonador" es el nivel. Sin pretender medir dificultad (subjetiva y dependiente del orden de componentes), se asigna un puntaje a cada tipo de geometría: **−1,0** huecos, enemigos y caídas; **−0,5** resortes y *stompers*; **+0,5** plataformas móviles; **+1,0** saltos sin hueco. La indulgencia total es el promedio normalizado.

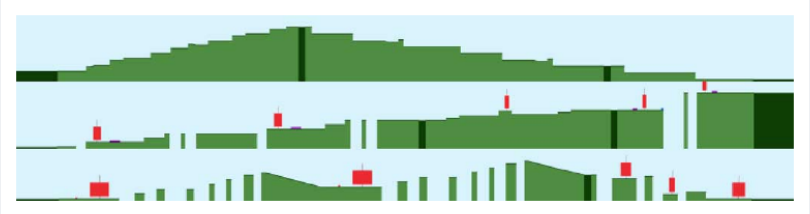


Figura 16. Indulgencia decreciente: 1 (arriba), 0,5 (medio) y 0,01 (abajo).

C. Métrica de distancia entre grupos de ritmo

Para comparar grupos de ritmo por similitud geométrica se usa la **distancia de edición de Levenshtein** (normalizada para reducir el efecto de la longitud) sobre vectores que codifican la geometría de cada beat (Tabla II). Inserción y borrado cuestan 2; la sustitución parte de 0 y suma +1 por agregar/quitar un hueco, +1 por plataforma móvil, +1 por cada paso de cambio vertical, +1 por aparición de enemigo o *stomper* y +0,5 por convertir enemigo↔stomper. Así se detectan patrones emergentes (p. ej. escaleras).

Tabla II. Abreviaturas de la geometría de nivel.

Código	Geometría
GA	Hueco hacia una plataforma más alta
MA	Plataforma móvil ascendente
A	Salto a plataforma más alta, sin hueco
GS	Hueco hacia plataforma de igual altura
MS	Plataforma móvil horizontal
GD	Hueco hacia plataforma más baja
MD	Plataforma móvil descendente
D	Salto a plataforma más baja, sin hueco
E	Enemigo
ST	*Stomper*

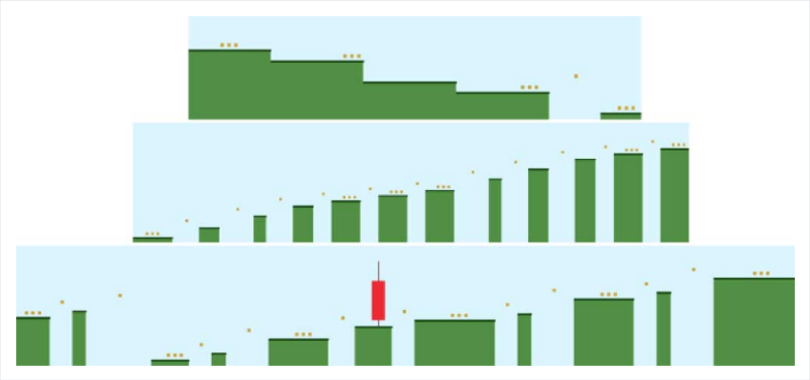


Figura 17. Distancias normalizadas entre tres grupos: d(A,B)=0,91; d(B,C)=0,20; d(A,C)=0,85. B y C son cercanos pese a tener distinta longitud: comparten patrón.

D. Rango expresivo

Se describe de dos formas complementarias: agrupando niveles completos en una grilla de linealidad × indulgencia, y haciendo clustering de grupos de ritmo para ver patrones emergentes.

1) Análisis de niveles

Con 10 000 niveles generados (todos los componentes con igual peso), el espacio está claramente **sesgado hacia niveles lineales** y levemente hacia niveles **poco indulgentes** (Fig. 18). El segundo sesgo se explica porque hay más componentes "no indulgentes". El primero es más interesante: proviene de un detalle de implementación añadido tarde (al elegir un componente, aumenta un poco la probabilidad de volver a elegirlo, para crear patrones repetidos como en Mario). Eso hace que los cambios de altura se "apilen" en tramos lineales: un efecto secundario no buscado, y un buen ejemplo de por qué conviene analizar el rango expresivo.

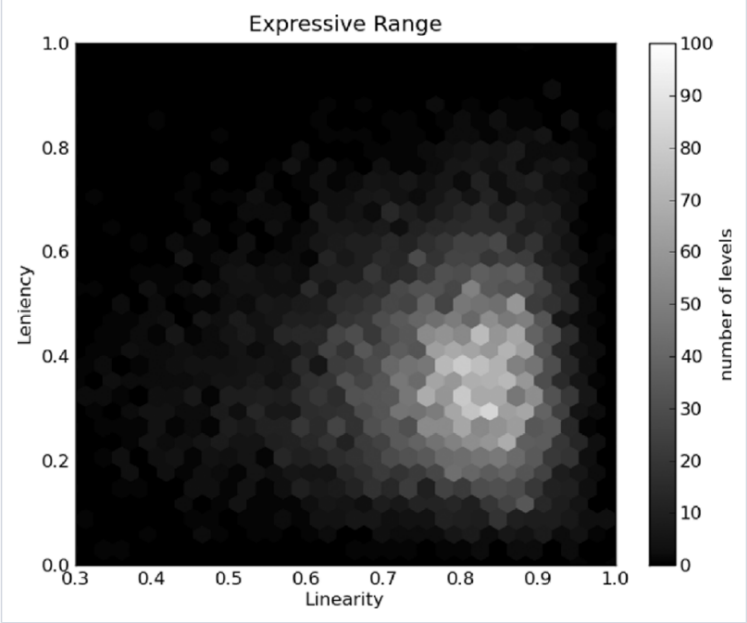


Figura 18. Rango expresivo de Launchpad (linealidad en X, de 0,3 a 1,0; indulgencia en Y, de 0 a 1). Cada hexágono cuenta los niveles con esos puntajes: más claro = más niveles.

Tipo de ritmo (Fig. 19). El ritmo regular da la mayor variación, sin un pico marcado. *Swing* y aleatorio son más restrictivos y empujan hacia niveles lineales y poco indulgentes; la hipótesis es que dejan menos tiempo para cada salto, por lo que la física descarta geometrías (en *swing* aparecen menos caídas, resortes y plataformas móviles).

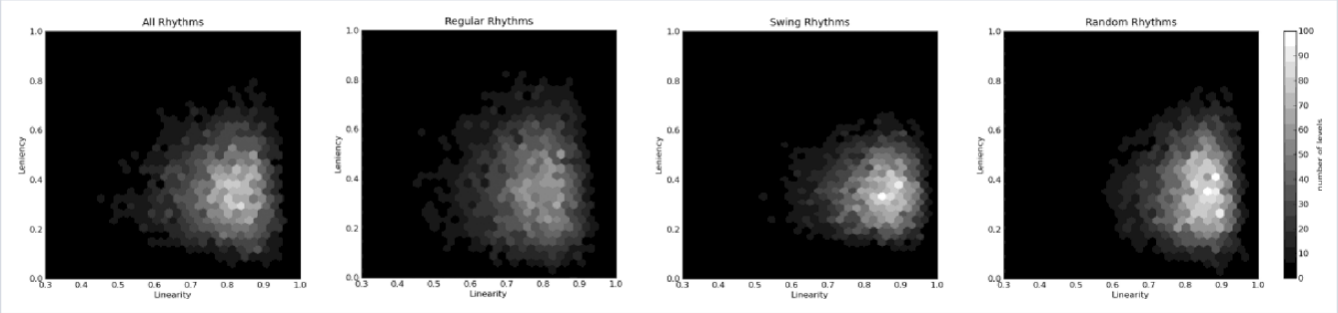


Figura 19. Variando el tipo de ritmo: (A) todos, (B) solo regular, (C) solo swing, (D) solo aleatorio.

Densidad (Fig. 20). No afecta notablemente la indulgencia, pero densidades altas producen niveles más lineales (más acciones → más repeticiones del mismo componente). **Longitud (Fig. 21)**: resultados similares.

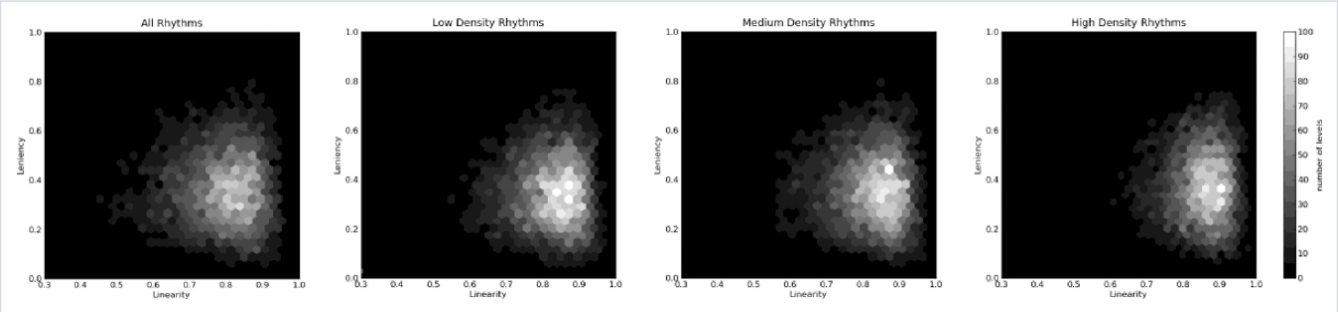


Figura 20. Variando la densidad: (A) todas, (B) baja, (C) media, (D) alta.

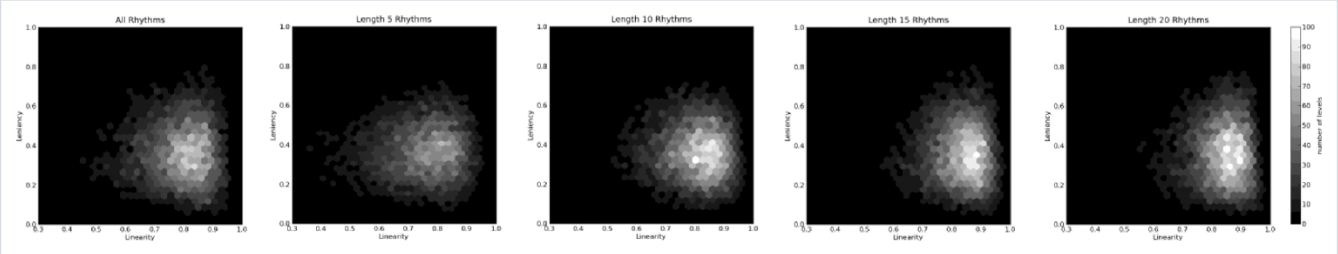


Figura 21. Variando la longitud: (A) todas, (B) 5 s, (C) 10 s, (D) 15 s, (E) 20 s.

2) Análisis de grupos de ritmo

Se usa **clustering jerárquico aglomerativo** sobre 4000 grupos (1000 por longitud). Las hojas son grupos con distancia cero entre sí (no necesariamente idénticos, Fig. 22); en cada iteración se unen los dos clusters cuyos miembros más disímiles están más cerca (enlace completo). Como un dendrograma es difícil de navegar, se visualiza con un **treemap** interactivo (Fig. 23): el área es proporcional al tamaño del cluster, con barra lateral de propiedades del ritmo y miniaturas de los grupos.

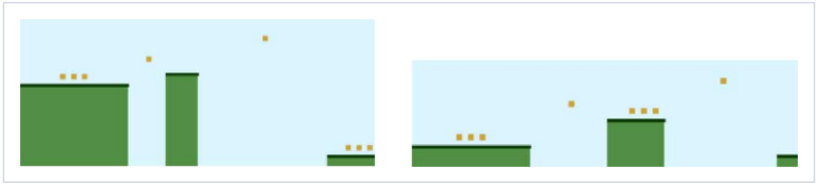


Figura 22. Dos grupos con distancia cero: no son idénticos, pero comparten el patrón de subir saltando un hueco y luego bajar saltando otro.

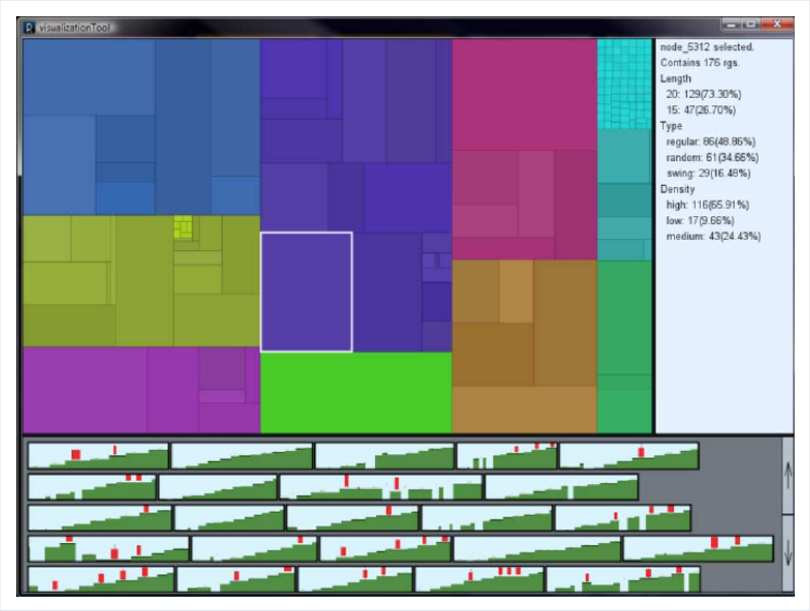


Figura 23. Herramienta de visualización de clusters (treemap navegable; abajo, los grupos de ritmo del cluster seleccionado).

La herramienta reveló más sesgos: los clusters más grandes tienen **muchos stompers** y abundan las **escaleras ascendentes**. Las escaleras refuerzan la hipótesis de que los tiempos de salto cortos eliminan saltos hacia abajo. El exceso de stompers se debe a que el patrón espera–movimiento–espera (plataforma móvil) también contiene un espera–movimiento (stomper), así que el stomper puede elegirse en ambos casos.

VI. Discusión y conclusiones

Los autores consideraron inicialmente un enfoque **basado en dificultad** (asignar dificultad a "idiomas" grandes como "saltar sobre un enemigo móvil" y ensamblarlos), pero lo descartaron por dar menos variedad y ser menos extensible. El enfoque por ritmo trabaja con los componentes más básicos y respeta la importancia estructural del ritmo; resulta más simple construir niveles con una estructura bien definida y analizar su dificultad después (como hace **Polymorph**).

También eligieron **generar-y-probar** en lugar de una búsqueda sofisticada para ensamblar grupos: permite al diseñador explorar rápidamente muchas opciones que cumplan criterios globales. Si los criterios fueran más finos (p. ej. geometría específica en ciertas regiones), una búsqueda sería más apropiada, porque el número de candidatos necesarios crecería demasiado.

Trabajo futuro: añadir nuevos verbos (p. ej. "disparar") y su geometría, ajustando el generador de ritmo (componentes largos como un *loop* de Sonic saldrían poco); disparadores que cambian reglas o física en tiempo de ejecución (como *Shift*) son difíciles con gramáticas, porque rompen la garantía de jugabilidad local; y extender la idea a otros géneros. Más que la técnica, lo importante es **razonar sobre las acciones esperadas del jugador**: beats en plataformas, misiones o habilidades en otros géneros.

Conclusiones

Launchpad genera niveles de plataformas 2D a partir de una comprensión formal del diseño de niveles, da al diseñador control mediante parámetros con un efecto claro, y garantiza jugabilidad. El paper además propone un método para analizar y visualizar la expresividad de un generador, y muestra que Launchpad tiene un amplio rango expresivo dentro de su dominio.

- **Fortaleza:** separar ritmo y geometría asegura el ritmo deseado y la jugabilidad.
- **Fortaleza:** mínima autoría manual (componentes básicos, no trozos de niveles).
- **Limitación:** solo niveles de una celda (sin caminos múltiples, secretos ni disparadores).
- **Lección metodológica:** el análisis de rango expresivo destapó sesgos (linealidad, stompers, escaleras) originados en detalles de implementación.